

02 - Odoo Deployment: Docker Image

LIONGATE SARL - Technical Runbook

Version 1.0 - Initial release

Table of Contents

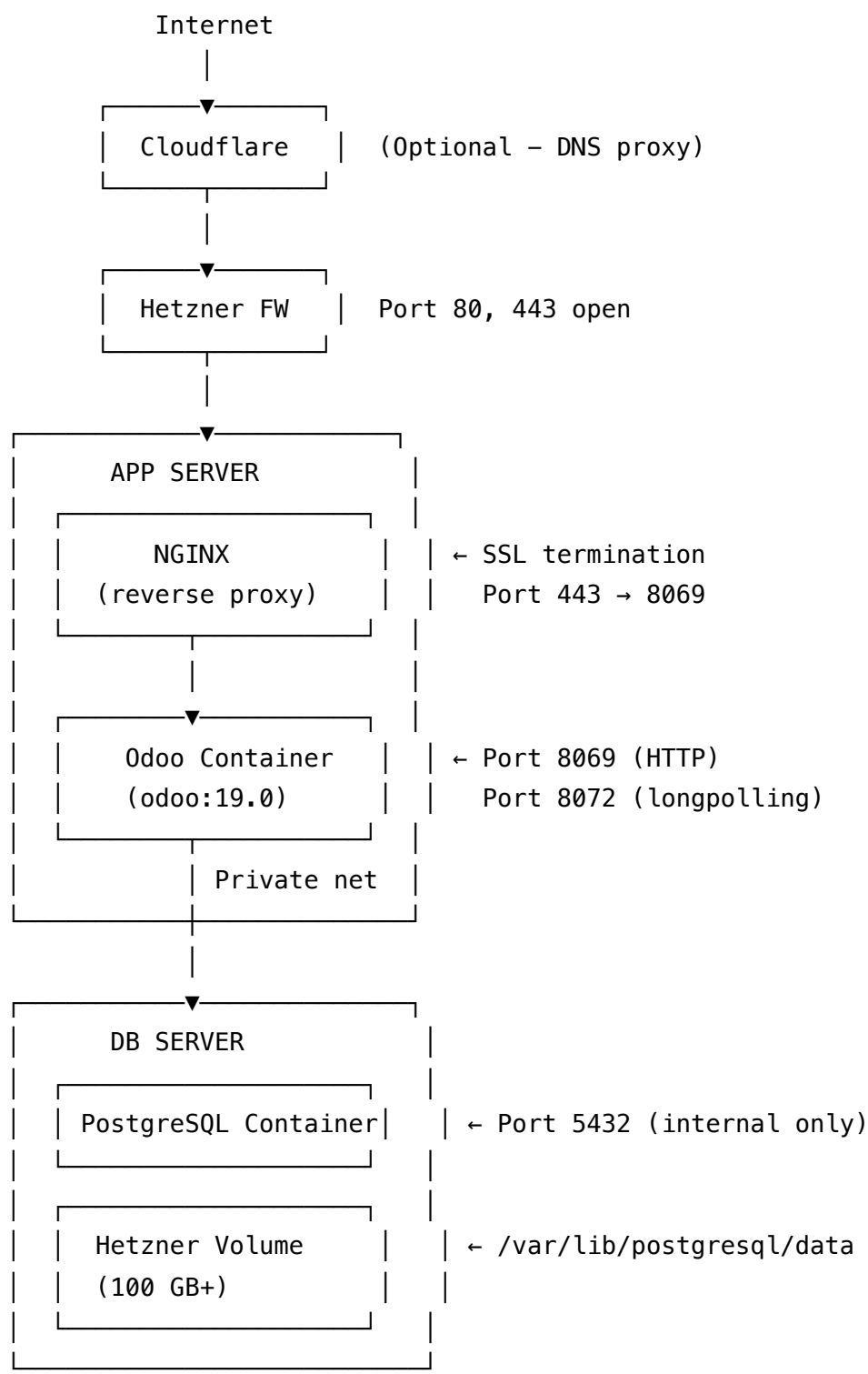
1. [Overview & Architecture](#)
2. [Version Assumptions](#)
3. [Local Development Deployment](#)
4. [Hetzner Server Provisioning](#)
5. [OS Preparation](#)
6. [Docker Installation](#)
7. [Container Architecture & Docker Compose](#)
8. [PostgreSQL Setup](#)
9. [Odoo Configuration](#)
10. [Volumes & Persistence](#)
11. [Reverse Proxy with Nginx](#)
12. [SSL/TLS Configuration](#)
13. [Domain Attachment](#)
14. [Email / SMTP Configuration](#)
15. [Worker & Performance Configuration](#)
16. [Security Hardening](#)
17. [Backup Strategy](#)
18. [Logging & Monitoring](#)
19. [Cron Jobs & Scheduled Actions](#)
20. [Upgrade Path](#)
21. [Recovery & Restore](#)
22. [Testing & Validation](#)
23. [Production Deployment Checklist](#)

Overview & Architecture

What This Document Covers

This document describes how to deploy Odoo using the official Docker image in a production-grade environment on Hetzner Cloud, connected to a domain from United Domains.

Architecture Overview



Technology Stack

Component	Technology	Version
Application	Odoo	17.0 (Community or Enterprise)

Component	Technology	Version
Runtime	Docker + Docker Compose	Docker 24+, Compose v2
Database	PostgreSQL	16.x
Reverse proxy	Nginx	1.24+
SSL	Let's Encrypt / Certbot	Latest
OS	Ubuntu	22.04 LTS
Monitoring	Prometheus + Grafana	Latest
Backups	pg_dump + rsync	System tools

Version Assumptions

Item	Value
Odoo version	19.0 (adjust tag for 17.0 or 18.0)
PostgreSQL version	16
Python version	Managed by Docker image (3.11 inside odoo:19.0)
Docker image	<code>odoo:19.0</code> (official Docker Hub image)
Ubuntu	22.04 LTS
Architecture	amd64 (Hetzner Cloud standard)

Important: The Odoo Docker image bundles its own Python runtime. You do not manage Python directly - this is one of the key advantages of the Docker approach.

Local Development Deployment

Prerequisites

- Docker Desktop (Mac/Windows) or Docker Engine (Linux)

- Docker Compose v2 (docker compose not docker-compose)
- Git

Directory Structure

```
odoo-local/  
├─ docker-compose.yml  
├─ config/  
│   └─ odoo.conf  
├─ addons/  
│   └─ custom/          ← your custom modules go here  
└─ .env
```

.env (local)

```
POSTGRES_DB=odoo  
POSTGRES_USER=odoo  
POSTGRES_PASSWORD=odoo_dev_password_change_me  
ODOO_DB=odoo  
ODOO_ADMIN_PASSWD=admin_master_password
```

docker-compose.yml (local)

version: "3.8"

services:

db:

image: postgres:16

environment:

POSTGRES_DB: \${POSTGRES_DB}

POSTGRES_USER: \${POSTGRES_USER}

POSTGRES_PASSWORD: \${POSTGRES_PASSWORD}

volumes:

- pgdata:/var/lib/postgresql/data

ports:

- "5432:5432"

healthcheck:

test: ["CMD-SHELL", "pg_isready -U \${POSTGRES_USER}"]

interval: 10s

timeout: 5s

retries: 5

odoo:

image: odoo:19.0

depends_on:

db:

condition: service_healthy

environment:

HOST: db

PORT: 5432

USER: \${POSTGRES_USER}

PASSWORD: \${POSTGRES_PASSWORD}

volumes:

- odoodata:/var/lib/odoo

- ./config:/etc/odoo

- ./addons:/mnt/extra-addons

ports:

- "8069:8069"

- "8072:8072"

command: -- --dev=all

volumes:

```
pgdata:
odoodata:
```

config/odoo.conf (local)

```
[options]
addons_path = /mnt/extra-addons,/usr/lib/python3/dist-packages/odoo/addons
data_dir = /var/lib/odoo
admin_passwd = admin_master_password
db_host = db
db_port = 5432
db_user = odoo
db_password = odoo_dev_password_change_me
db_name = odoo
log_level = debug
```

Start Local Environment

```
cd odoo-local
docker compose up -d
# Access: http://localhost:8069
# Initial DB setup takes 1-3 minutes on first run
```

Hetzner Server Provisioning

Recommended Server Configuration (Production)

Resource	Specification
App server	CX42 (8 vCPU, 16 GB RAM, 160 GB NVMe)
DB server	CX32 (4 vCPU, 8 GB RAM, 80 GB NVMe)
DB Volume	100 GB Hetzner Volume (attached to DB server)
Backup	Storage Box BX31 (1 TB)
Floating IP	1 (assigned to app server)

Resource	Specification
Private network	10.0.0.0/16
Region	Nuremberg (nbg1) or Helsinki (hel1)

Provision via Hetzner Console or CLI

```
# Install hcloud CLI
brew install hcloud # macOS
# OR download from https://github.com/hetznercloud/cli/releases

hcloud context create liongate
# Enter your Hetzner API token

# Create private network
hcloud network create --name liongate-net --ip-range 10.0.0.0/16
hcloud network add-subnet liongate-net --type cloud --network-zone eu-central --ip-rang

# Create app server
hcloud server create \
  --name odoo-app \
  --type cx42 \
  --image ubuntu-22.04 \
  --ssh-key your-ssh-key-name \
  --network liongate-net \
  --location nbg1

# Create DB server
hcloud server create \
  --name odoo-db \
  --type cx32 \
  --image ubuntu-22.04 \
  --ssh-key your-ssh-key-name \
  --network liongate-net \
  --location nbg1

# Create and attach volume for DB
hcloud volume create --name odoo-pgdata --size 100 --server odoo-db

# Create floating IP
hcloud floating-ip create --type ipv4 --home-location nbg1 --name odoo-fip
hcloud floating-ip assign odoo-fip odoo-app

# Create firewall
hcloud firewall create --name odoo-fw
hcloud firewall add-rule odoo-fw --direction in --protocol tcp --port 22 --source-ips "
hcloud firewall add-rule odoo-fw --direction in --protocol tcp --port 80 --source-ips "
```

```
hcloud firewall add-rule odoo-fw --direction in --protocol tcp --port 443 --source-ips  
hcloud firewall apply-to-server odoo-fw --server odoo-app
```

OS Preparation

Run the following on **both servers** after provisioning.

```
# SSH to each server
ssh root@<server-ip>

# Update system
apt update && apt upgrade -y

# Set hostname
hostnamectl set-hostname odoo-app # or odoo-db on DB server

# Set timezone
timedatectl set-timezone Africa/Douala

# Install essential tools
apt install -y \
    curl wget git unzip htop vim \
    net-tools ufw fail2ban \
    ca-certificates gnupg lsb-release

# Configure UFW firewall (app server)
ufw default deny incoming
ufw default allow outgoing
ufw allow from YOUR_OFFICE_IP to any port 22
ufw allow 80/tcp
ufw allow 443/tcp
ufw --force enable

# Configure UFW (DB server - only allow app server)
ufw default deny incoming
ufw default allow outgoing
ufw allow from 10.0.1.0/24 to any port 5432 # Only from private network
ufw allow from YOUR_OFFICE_IP to any port 22
ufw --force enable

# Configure fail2ban
systemctl enable fail2ban
systemctl start fail2ban

# Create deploy user (non-root)
useradd -m -s /bin/bash deploy
usermod -aG sudo deploy
mkdir -p /home/deploy/.ssh
cp /root/.ssh/authorized_keys /home/deploy/.ssh/
chown -R deploy:deploy /home/deploy/.ssh
```

```
chmod 700 /home/deploy/.ssh
chmod 600 /home/deploy/.ssh/authorized_keys
```

Mount DB Volume (on DB server)

```
# Find the volume device
lsblk
# Usually /dev/sdb or /dev/disk/by-id/...

# Format (first time only - WARNING: destroys data)
mkfs.ext4 /dev/sdb

# Create mount point
mkdir -p /mnt/pgdata

# Mount
mount /dev/sdb /mnt/pgdata

# Add to fstab for persistence
echo "/dev/sdb /mnt/pgdata ext4 defaults 0 0" >> /etc/fstab

# Verify
df -h | grep pgdata
```

Docker Installation

Run on **both servers**.

```
# Add Docker GPG key
install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
    gpg --dearmor -o /etc/apt/keyrings/docker.gpg
chmod a+r /etc/apt/keyrings/docker.gpg

# Add Docker repository
echo \
    "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
    https://download.docker.com/linux/ubuntu \
    $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
    tee /etc/apt/sources.list.d/docker.list > /dev/null

# Install Docker
apt update
apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose

# Enable and start Docker
systemctl enable docker
systemctl start docker

# Add deploy user to docker group
usermod -aG docker deploy

# Verify
docker --version
docker compose version
```

Container Architecture & Docker Compose

Directory Structure (App Server)

```
/opt/odoo/
├── docker-compose.yml
├── .env                ← NEVER commit to git
├── config/
│   └── odoo.conf
├── addons/
│   └── custom/        ← Custom modules
├── nginx/
│   ├── nginx.conf
│   └── ssl/
│       ├── fullchain.pem
│       └── privkey.pem
└── scripts/
    ├── backup.sh
    └── restore.sh
```

docker-compose.yml (Production - App Server)

```
version: "3.8"

networks:
  odoo_internal:
    driver: bridge

services:
  odoo:
    image: odoo:19.0
    container_name: odoo_app
    restart: unless-stopped
    depends_on:
      - odoo_init
    environment:
      HOST: ${DB_HOST}
      PORT: ${DB_PORT}
      USER: ${DB_USER}
      PASSWORD: ${DB_PASSWORD}
    volumes:
      - odoo_filestore:/var/lib/odoo
      - ./config:/etc/odoo:ro
      - ./addons/custom:/mnt/extra-addons:ro
    ports:
      - "127.0.0.1:8069:8069"
      - "127.0.0.1:8072:8072"
    networks:
      - odoo_internal
    command: --workers=4 --max-cron-threads=2 --limit-memory-hard=2684354560 --limit-me
    logging:
      driver: "json-file"
      options:
        max-size: "100m"
        max-file: "5"
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8069/web/health"]
      interval: 30s
      timeout: 10s
      retries: 5
      start_period: 60s

  odoo_init:
```

```

image: odoo:19.0
container_name: odoo_init
restart: "no"
environment:
  HOST: ${DB_HOST}
  PORT: ${DB_PORT}
  USER: ${DB_USER}
  PASSWORD: ${DB_PASSWORD}
volumes:
  - odoo_filestore:/var/lib/odoo
  - ./config:/etc/odoo:ro
  - ./addons/custom:/mnt/extra-addons:ro
command: odoo --stop-after-init -i base
profiles:
  - init

```

```

volumes:
  odoo_filestore:
    driver: local
    driver_opts:
      type: none
      o: bind
      device: /opt/odoo/filestore

```

.env (Production)

```

# Database
DB_HOST=10.0.1.20           # Private IP of DB server
DB_PORT=5432
DB_NAME=odoo_production
DB_USER=odoo_prod
DB_PASSWORD=REPLACE_WITH_STRONG_SECRET

# Odoo
ODOO_ADMIN_PASSWD=REPLACE_WITH_STRONG_MASTER_PASSWORD

# App
APP_DOMAIN=erp.clientname.com

```

Security: Store `.env` with `chmod 600`. Use a secrets manager (Vault or GitHub Secrets) for CI/CD.

PostgreSQL Setup

On the **DB Server**, run PostgreSQL in Docker with data mounted to the Hetzner Volume.

`docker-compose.yml` (DB Server)

```
version: "3.8"

services:
  postgres:
    image: postgres:16
    container_name: postgres_odoo
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      PGDATA: /var/lib/postgresql/data
    volumes:
      - /mnt/pgdata:/var/lib/postgresql/data
      - ./init:/docker-entrypoint-initdb.d
    ports:
      - "10.0.1.20:5432:5432" # Only listen on private network IP
    shm_size: 256mb
    logging:
      driver: "json-file"
      options:
        max-size: "100m"
        max-file: "5"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 10s
      timeout: 5s
      retries: 5
```

PostgreSQL Performance Tuning

Add a custom `postgresql.conf` override:

```
# /mnt/pgdata/postgresql.conf additions (append or override)
# Tune for 8 GB RAM DB server
```

```
max_connections = 100
shared_buffers = 2GB
effective_cache_size = 6GB
maintenance_work_mem = 512MB
checkpoint_completion_target = 0.9
wal_buffers = 64MB
default_statistics_target = 100
random_page_cost = 1.1
effective_io_concurrency = 200
work_mem = 52428kB
huge_pages = off
min_wal_size = 1GB
max_wal_size = 4GB
max_worker_processes = 8
max_parallel_workers_per_gather = 4
max_parallel_workers = 8
max_parallel_maintenance_workers = 4
```

Odoo Configuration

config/odoo.conf (Production)

```
[options]
; --- Paths ---
addons_path = /mnt/extra-addons,/usr/lib/python3/dist-packages/odoo/addons
data_dir = /var/lib/odoo

; --- Database ---
db_host = 10.0.1.20
db_port = 5432
db_user = odoo_prod
db_password = REPLACE_WITH_STRONG_SECRET
db_name = odoo_production
db_maxconn = 64

; --- Security ---
admin_passwd = REPLACE_WITH_STRONG_MASTER_PASSWORD
list_db = False

; --- HTTP ---
http_port = 8069
http_interface = 0.0.0.0
longpolling_port = 8072

; --- Proxy ---
proxy_mode = True

; --- Workers ---
workers = 4
max_cron_threads = 2

; --- Memory limits ---
limit_memory_hard = 2684354560 ; 2.5 GB
limit_memory_soft = 2147483648 ; 2.0 GB
limit_request = 8192
limit_time_cpu = 600
limit_time_real = 1200
limit_time_real_cron = -1

; --- Logging ---
```

```
log_level = warn
log_handler = :WARNING
logfile = /var/lib/odoo/odoo.log
log_db = False

; --- Email ---
smtp_server = smtp.provider.com
smtp_port = 587
smtp_ssl = False
smtp_starttls = True
smtp_user = noreply@clientname.com
smtp_password = REPLACE_WITH_SMTP_SECRET
email_from = noreply@clientname.com

; --- Misc ---
without_demo = True
server_wide_modules = base,web
```

Volumes & Persistence

Critical Volumes

Volume	Path in Container	Purpose	Must Persist?
odoo_filestore	/var/lib/odoo	Attachments, sessions	Yes
PostgreSQL data	/var/lib/postgresql/data	All data	Yes
Custom addons	/mnt/extra-addons	Code (read-only OK)	No (rebuild)
Config	/etc/odoo	odoo.conf	No (in Git)

Filestore on App Server

```
# Create filestore directory
mkdir -p /opt/odoo/filestore
chown -R 101:101 /opt/odoo/filestore    # Odoo UID in official image

# If growing large, move to Hetzner Volume
mkdir -p /mnt/odoo-filestore
mount /dev/sdc /mnt/odoo-filestore
ln -s /mnt/odoo-filestore /opt/odoo/filestore
```

Reverse Proxy with Nginx

Install Nginx (App Server - bare metal, not Docker)

```
apt install -y nginx
```

/etc/nginx/sites-available/odoo

```
upstream odoo {
    server 127.0.0.1:8069 weight=1 fail_timeout=0;
}

upstream odoo_chat {
    server 127.0.0.1:8072 weight=1 fail_timeout=0;
}

# HTTP → HTTPS redirect
server {
    listen 80;
    server_name erp.clientname.com;
    return 301 https://$host$request_uri;
}

# HTTPS server
server {
    listen 443 ssl http2;
    server_name erp.clientname.com;

    ssl_certificate      /etc/letsencrypt/live/erp.clientname.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/erp.clientname.com/privkey.pem;
    ssl_session_timeout 30m;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-A
    ssl_prefer_server_ciphers off;
    ssl_stapling on;
    ssl_stapling_verify on;
    add_header Strict-Transport-Security "max-age=63072000" always;

    # Security headers
    add_header X-Frame-Options SAMEORIGIN;
    add_header X-Content-Type-Options nosniff;
    add_header X-XSS-Protection "1; mode=block";
    add_header Referrer-Policy "strict-origin-when-cross-origin";

    # Logging
    access_log /var/log/nginx/odoo_access.log;
    error_log /var/log/nginx/odoo_error.log;

    # Proxy buffers
```

```

proxy_read_timeout 720s;
proxy_connect_timeout 720s;
proxy_send_timeout 720s;
proxy_buffers 16 64k;
proxy_buffer_size 128k;

# Gzip
gzip on;
gzip_types text/css text/plain application/json application/javascript;

# Max upload size (for Odoo attachments)
client_max_body_size 100m;

# Longpolling (Odoo chat / bus)
location /web/websocket {
    proxy_pass http://odoo_chat;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "Upgrade";
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

location /longpolling {
    proxy_pass http://odoo_chat;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

# Static files – serve directly for performance
location ~* /web/static/ {
    proxy_cache_valid 200 90m;
    proxy_buffering on;
    expires 864000;
    proxy_pass http://odoo;
    proxy_set_header Host $host;
}

# Main proxy
location / {

```

```
    proxy_pass http://odoo;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_redirect off;
}

}

# Enable site
ln -s /etc/nginx/sites-available/odoo /etc/nginx/sites-enabled/odoo
nginx -t
systemctl reload nginx
```

SSL/TLS Configuration

```
# Install Certbot
apt install -y certbot python3-certbot-nginx

# Obtain certificate
certbot --nginx -d erp.clientname.com \
    --non-interactive \
    --agree-tos \
    --email admin@liongate.com \
    --redirect

# Verify auto-renewal
certbot renew --dry-run

# Auto-renewal is handled by systemd timer (installed by certbot)
systemctl status certbot.timer
```

Domain Attachment

Steps in United Domains

1. Log in to United Domains control panel
2. Navigate to your domain → **DNS Management**
3. Add an **A record**:
 - Name: erp (for erp.clientname.com)
 - Type: A
 - Value: <Hetzner Floating IP>
 - TTL: 3600
4. Save and wait for DNS propagation (5–30 minutes typically)

Verify DNS

```
# Check DNS propagation
```

```
dig erp.clientname.com +short
```

```
nslookup erp.clientname.com
```

```
# Test SSL
```

```
curl -I https://erp.clientname.com
```

```
openssl s_client -connect erp.clientname.com:443 -servername erp.clientname.com
```

Email / SMTP Configuration

Recommended Email Providers

Provider	Free tier	SMTP host	Port
Brevo (Sendinblue)	300 emails/day	smtp-relay.brevo.com	587
Mailgun	100 emails/day	smtp.mailgun.org	587
Postmark	Paid only	smtp.postmarkapp.com	587
Gmail SMTP	Personal only	smtp.gmail.com	587

Configure in Odoo

Settings → Technical → Email → Outgoing Mail Servers

- SMTP Server: smtp-relay.brevo.com
- SMTP Port: 587
- Connection Security: STARTTLS
- Username: your-brevo-smtp-login
- Password: your-smtp-api-key
- From: noreply@clientname.com

Test connection after saving.

Configure Incoming Mail (optional)

Settings → Technical → Email → Incoming Mail Servers

- Server Type: IMAP or POP3
- Server: imap.yourmailprovider.com
- Port: 993 (IMAP SSL)
- Username/Password: mailbox credentials

Worker & Performance Configuration

Worker Sizing Formula

$\text{workers} = (\text{CPU cores} \times 2) + 1$

$\text{cron_threads} = 2$ (always)

For CX42 (8 vCPU):

$\text{workers} = (8 \times 2) + 1 = 17$ ← theoretical max

Recommended: workers = 6–8 (leave headroom)

$\text{max_cron_threads} = 2$

Memory per worker: ~300–500 MB (Odoo 17)

Total memory needed: $(\text{workers} \times 400 \text{ MB}) + 2 \text{ GB OS} = \sim 5.2 \text{ GB}$ for 8 workers

CX42 (16 GB RAM) supports this comfortably.

Worker Configuration in `odoo.conf`

```
workers = 6
max_cron_threads = 2
limit_memory_soft = 1610612736 ; 1.5 GB per worker
limit_memory_hard = 2147483648 ; 2 GB hard limit
limit_time_cpu = 600
limit_time_real = 1200
```

Security Hardening

Odoo-Specific

```
# 1. Disable database listing (odoo.conf)
list_db = False

# 2. Set strong master password (admin_passwd in odoo.conf)
# Generate: openssl rand -base64 32

# 3. Disable developer mode in production
# Odoo → Settings → Developer Tools → Deactivate developer mode

# 4. Restrict database manager
# Only accessible from internal network (via Nginx IP restriction)

# 5. Enable 2FA for admin users
# Settings → Users → Enable Two-Factor Authentication
```

Nginx Security (already in config above)

```
# Already configured:
# - HSTS header
# - X-Frame-Options: SAMEORIGIN
# - X-Content-Type-Options: nosniff
# - TLS 1.2/1.3 only
# - Strong cipher suite
```

Server Hardening

```
# Disable root SSH login
sed -i 's/PermitRootLogin yes/PermitRootLogin no/' /etc/ssh/sshd_config
sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config
systemctl restart sshd
```

```
# Install unattended upgrades (auto-patch security)
apt install -y unattended-upgrades
dpkg-reconfigure -plow unattended-upgrades
```

```
# Audit running services
ss -tlnp
```

```
# Docker security – don't expose sockets
chmod 660 /var/run/docker.sock
```

Backup Strategy

Automated Backup Script

```
#!/bin/bash
# /opt/odoo/scripts/backup.sh

set -euo pipefail

TIMESTAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_DIR="/opt/odoo/backups"
ODOO_DB="odoo_production"
DB_HOST="10.0.1.20"
DB_USER="odoo_prod"
STORAGE_BOX_HOST="uXXXXXX.your-storagebox.de"
STORAGE_BOX_USER="uXXXXXX"
RETENTION_DAYS=7

mkdir -p "$BACKUP_DIR"

echo "=== Odoo Backup: $TIMESTAMP ==="

# 1. Dump PostgreSQL database
echo "Dumping database..."
PGPASSWORD="$DB_PASSWORD" pg_dump \
    -h "$DB_HOST" \
    -U "$DB_USER" \
    -d "$ODOO_DB" \
    -Fc \
    -f "$BACKUP_DIR/db_${TIMESTAMP}.dump"

# 2. Backup filestore
echo "Backing up filestore..."
tar -czf "$BACKUP_DIR/filestore_${TIMESTAMP}.tar.gz" \
    /opt/odoo/filestore/

# 3. Backup config
cp /opt/odoo/config/odoo.conf "$BACKUP_DIR/odoo_conf_${TIMESTAMP}.conf"

# 4. Upload to Storage Box (rsync over SSH)
echo "Uploading to Storage Box..."
rsync -avz --progress \
```

```

"$BACKUP_DIR/" \
"${STORAGE_BOX_USER}@${STORAGE_BOX_HOST}:/backups/odoo/" \
-e "ssh -i /root/.ssh/storagebox_rsa -p 23"

# 5. Delete old local backups
find "$BACKUP_DIR" -name "*.dump" -mtime +$RETENTION_DAYS -delete
find "$BACKUP_DIR" -name "*.tar.gz" -mtime +$RETENTION_DAYS -delete

echo "=== Backup complete: $TIMESTAMP ==="

# Install cron job
chmod +x /opt/odoo/scripts/backup.sh
crontab -e
# Add:
# 0 2 * * * /opt/odoo/scripts/backup.sh >> /var/log/odoo-backup.log 2>&1

```

Logging & Monitoring

Log Files

Log	Location	Content
Odoo application	/var/lib/odoo/odoo.log (in container)	App errors, access
Nginx access	/var/log/nginx/odoo_access.log	HTTP requests
Nginx error	/var/log/nginx/odoo_error.log	Proxy errors
PostgreSQL	Docker JSON logs	DB queries, errors
System	/var/log/syslog	OS events

View Logs

```
# Odoo container logs
docker logs odoo_app --tail=100 -f

# Nginx
tail -f /var/log/nginx/odoo_error.log

# All containers
docker compose -f /opt/odoo/docker-compose.yml logs -f
```

Basic Uptime Monitoring

```
# Install netdata (lightweight, free)
bash <(curl -Ss https://my-netdata.io/kickstart.sh) --non-interactive

# Or use UptimeRobot (free external monitoring)
# Create monitor: https://erp.clientname.com/web/health
# Alert on: HTTP != 200 for 5 minutes
```

Cron Jobs & Scheduled Actions

Odoo manages its own internal cron jobs (Scheduled Actions) through the UI.

Settings → Technical → Automation → Scheduled Actions

Key system crons:

- Auto Vacuum (daily)
- Mail Queue cleanup (hourly)
- Discuss cleanup (daily)
- Action Server cleanup (daily)

External cron (backup) is managed in system crontab as shown in the Backup section.

Upgrade Path

Minor version upgrades (e.g., 17.0.1 → 17.0.2)

```
cd /opt/odoo

# Pull new Docker image
docker compose pull

# Stop Odoo (not DB)
docker compose stop odoo

# Update DB (run upgrade)
docker compose run --rm odoo odoo -u all --stop-after-init

# Start Odoo
docker compose up -d odoo
```

Major version upgrades (e.g., 16.0 → 17.0)

Major upgrades require the official Odoo upgrade tool and are **destructive** to data structures. Always:

1. **Full backup** of DB and filestore
2. Test upgrade on a staging copy first
3. Use `odoo-upgrade` official tooling
4. Allow 2–8 hours of downtime
5. Test all critical workflows before going live

Recovery & Restore

Restore Database

```
# Download backup from Storage Box
rsync -avz \
    "${STORAGE_BOX_USER}@${STORAGE_BOX_HOST}:/backups/odoo/db_TIMESTAMP.dump" \
    /tmp/restore/

# Stop Odoo
cd /opt/odoo && docker compose stop odoo

# Drop and recreate DB (on DB server)
docker exec postgres_odoo psql -U odoo_prod -c "DROP DATABASE IF EXISTS odoo_production"
docker exec postgres_odoo psql -U odoo_prod -c "CREATE DATABASE odoo_production;"

# Restore
PGPASSWORD="${DB_PASSWORD}" pg_restore \
    -h 10.0.1.20 \
    -U odoo_prod \
    -d odoo_production \
    /tmp/restore/db_TIMESTAMP.dump

# Restore filestore
tar -xzf /tmp/restore/filestore_TIMESTAMP.tar.gz -C /

# Start Odoo
docker compose up -d odoo
```

Testing & Validation

Post-Deployment Validation

1. Health check

```
curl -s https://erp.clientname.com/web/health | python3 -m json.tool
```

2. SSL validation

```
curl -I https://erp.clientname.com
```

```
openssl s_client -connect erp.clientname.com:443 2>/dev/null | grep "Verify return code"
```

3. Container status

```
docker compose ps
```

4. Odoo login

→ Access https://erp.clientname.com

→ Log in with admin credentials

→ Verify modules installed

5. Database test

```
docker exec postgres_odoo psql -U odoo_prod -c "\l"
```

```
docker exec postgres_odoo psql -U odoo_prod -d odoo_production -c "SELECT count(*) FROM"
```

6. Send test email from Odoo

Settings → Technical → Email → Outgoing Mail Servers → Test Connection

7. Check logs

```
docker logs odoo_app --tail=50 | grep -i error
```

Production Deployment Checklist

Pre-Deployment

- ☐ Hetzner servers provisioned and accessible
- ☐ Private network configured between app and DB servers
- ☐ Firewall rules applied (only ports 80, 443, 22 from trusted IP)
- ☐ Docker installed on both servers
- ☐ DB Volume mounted and formatted

- ☐ DNS A record pointing to Floating IP
- ☐ `.env` file created with production secrets
- ☐ `odoo.conf` configured with production settings
- ☐ `list_db = False` set
- ☐ Strong `admin_passwd` set
- ☐ SMTP credentials configured

Deployment

- ☐ PostgreSQL container started on DB server
- ☐ DB connection verified from app server
- ☐ Odoo container started
- ☐ Odoo accessed on port 8069 (internal) - functional
- ☐ Initial database initialized
- ☐ Nginx configured and tested (`nginx -t`)
- ☐ Let's Encrypt certificate obtained
- ☐ HTTPS access verified
- ☐ Redirect from HTTP to HTTPS verified

Post-Deployment

- ☐ Admin user password changed from default
- ☐ Developer mode disabled
- ☐ `list_db = False` verified (try `/web/database/manager`)
- ☐ Backup script deployed and cron scheduled
- ☐ First backup tested manually
- ☐ Backup restore tested on test environment
- ☐ Monitoring set up (UptimeRobot minimum)
- ☐ Log rotation configured
- ☐ SSL auto-renewal verified (`certbot renew --dry-run`)
- ☐ Odoo scheduled actions reviewed in UI
- ☐ Email sending tested
- ☐ Fail2ban enabled on both servers
- ☐ SSH root login disabled

Document owner: LIONGATE SARL DevOps Team

Last updated: See Git history

Next review: After each Odoo version upgrade